

OPUS.NET TODO Application

Full chronological conversation transcript - architecture, ClientDomainRuntime, binary protocols, OPUSGuid, serialization, DistributedMiddleTier, and ObjectNetworkEngine contracts

1. USER

based on the OPUS.NET framework design in the notebook, give me a full multi-layer Task-based TODO app as before with client side domain model with an application class singleton and task objects within this singleton throughout the following layer as specified in this top to bottom construct:

1. ClientDomainRuntime
2. ClientBinaryProtocol
3. ServerBinaryProtocol
4. ServerFacade
5. ObjectNetworkEngine
6. DistributedMiddleTier
7. ObjectNetworkEngine
8. PhysicalStorage

2. ASSISTANT

Yes. Based on the OPUS.NET design we established, I would implement the TODO application as a reference application for the framework itself, not merely as a demo UI.

The essential architecture is:

```
WPF / UI

|
v
1. ClientDomainRuntime
| Client Application singleton
| Client Task objects
v
2. ClientBinaryProtocol
|
| TCP/IP binary messages
v
3. ServerBinaryProtocol
|
v
4. ServerFacade
| private Server Application domain model
v
5. ObjectNetworkEngine
|
v
6. DistributedMiddleTier
|
v
7. ObjectNetworkEngine
|
v
8. PhysicalStorage

GUID -> byte[]
```

There is one architectural adjustment I recommend: although the logical object hierarchy is Application -> Tasks throughout, the actual client objects should not be the server objects. The protocol carries DTO/message representations. That preserves the earlier OPUS.NET rule that the server-side domain model remains private.

1. Solution structure

OPUS.TODO.sln

OPUS.Core

OPUSGuid.cs

OPUS.TODO.Contracts

TaskData.cs

ApplicationData.cs

Commands.cs

Responses.cs

OPUS.TODO.ClientDomain

Application.cs

Task.cs

ClientDomainRuntime.cs

OPUS.TODO.ClientProtocol

ClientBinaryProtocol.cs

BinaryMessageSerializer.cs

OPUS.TODO.ServerProtocol

ServerBinaryProtocol.cs

TcpServer.cs

OPUS.TODO.ServerDomain

Application.cs

Task.cs

OPUS.TODO.ServerFacade

ServerFacade.cs

OPUS.ObjectNetwork

ObjectNetworkEngine.cs

ObjectSerializer.cs

OPUS.DistributedMiddleTier

DistributedMiddleTier.cs

OPUS.ObjectNetwork.StorageNode

ObjectNetworkEngine.cs

OPUS.PhysicalStorage

IObjectStore.cs

FileSystemObjectStore.cs

OPUS.TODO.Server

Program.cs

OPUS.TODO.Client

WPF application

For an initial implementation these can also be folders/namespaces inside fewer assemblies. The separation above shows where the OPUS.NET framework boundaries are.

2. Core OPUS GUID

I'll use OPUSGuid conceptually below. If your existing class is already working, keep it.

For readability, this implementation assumes:

```
public struct OPUSGuid
{
    private readonly Guid _value;

    public OPUSGuid(Guid value)
    {
        _value = value;
    }
}
```

```

    }

    public static OPUSGuid Empty
    {
        get { return new OPUSGuid(Guid.Empty); }
    }

    public static OPUSGuid NewRandom128()
    {
        return new OPUSGuid(Guid.NewGuid());
    }

    public Guid ToGuid()
    {
        return _value;
    }

    public byte[] ToByteArray()
    {
        return _value.ToByteArray();
    }

    public static OPUSGuid FromByteArray(byte[] value)
    {
        return new OPUSGuid(new Guid(value));
    }

    public override string ToString()
    {
        return _value.ToString();
    }

    public override bool Equals(object obj)
    {
        if (!(obj is OPUSGuid))
            return false;

        return _value.Equals(((OPUSGuid)obj)._value);
    }

    public override int GetHashCode()
    {
        return _value.GetHashCode();
    }

    public static bool operator ==(OPUSGuid a, OPUSGuid b)
    {
        return a.Equals(b);
    }

    public static bool operator !=(OPUSGuid a, OPUSGuid b)
    {
        return !a.Equals(b);
    }
}

```

3. Shared protocol contracts

These are not domain objects. They are wire representations.

```

namespace OPUS.TODO.Contracts
{
    public sealed class TaskData
    {
        {
            public OPUSGuid Id { get; set; }
            public string Title { get; set; }
            public string Description { get; set; }
            public bool IsCompleted { get; set; }
        }
    }
}

namespace OPUS.TODO.Contracts

```

```

{
public sealed class ApplicationData
{
    public OPUSGuid Id { get; set; }
    public List<TaskData> Tasks { get; set; }
}

public ApplicationData()
{
    Tasks = new List<TaskData>();
}
}

Client Task -> TaskData -> TCP/IP -> TaskData -> Server Task

```

The server never hands its actual domain objects to the client.

```

4. Layer 1 - ClientDomainRuntime
namespace OPUS.TODO.ClientDomain
{
public sealed class Task
{
    public OPUSGuid Id { get; internal set; }
    public string Title { get; set; }
    public string Description { get; set; }
    public bool IsCompleted { get; set; }
}
}

```

```

internal Task(OPUSGuid id)
{
    Id = id;
}
}

namespace OPUS.TODO.ClientDomain
{

```

```

public sealed class Application
{
    private static readonly Application _instance = new Application();
}

public static Application Instance
{
    get { return _instance; }
}

public OPUSGuid Id { get; internal set; }
public ObservableCollection<Task> Tasks { get; private set; }
}

```

```

private Application()
{
    Tasks = new ObservableCollection<Task>();
}
}

```

Client application code accesses Application.Instance.Tasks.

ClientDomainRuntime service keeps the client domain model synchronized with the server, loading ApplicationData, creating TaskData, etc.

5. Protocol commands

```

public enum OperationCode : byte
{
    GetApplication = 1,
    CreateTask = 2,
    UpdateTask = 3,
    DeleteTask = 4
}

```

The first byte is reserved for read/write distinction:

```

0 = READ
1 = WRITE

```

Request:

BYTE 0 Lock mode

BYTE 1 Operation

INT32 Payload length

BYTE[] Payload

6. Layer 2 - ClientBinaryProtocol

ClientBinaryProtocol connects with TcpClient, writes the read/write flag, operation byte, payload length and payload, then

Initially one TCP connection per operation is recommended.

7. Binary serialization

Do not use BinaryFormatter. Use deterministic explicit serialization. Task fields are written in a known order: 16-byte OPUSGuid, explicit UTF-8 strings, and Boolean.

8. Layer 3 - ServerBinaryProtocol

ServerBinaryProtocol reverses the operation. It reads the write flag, operation, payload length and payload, acquires Reader

9. Server domain model

Server Task:

```
public sealed class Task
```

```
{
    public OPUSGuid Id { get; private set; }
    public string Title { get; set; }
    public string Description { get; set; }
    public bool IsCompleted { get; set; }
}
```

```
public Task(OPUSGuid id)
```

```
{
    Id = id;
}
```

Server Application singleton:

```
public sealed class Application
```

```
{
    private static readonly Application _instance = new Application();
    public static Application Instance { get { return _instance; } }

    public OPUSGuid Id { get; internal set; }
    public List<Task> Tasks { get; private set; }
}
```

```
private Application()
```

```
{
    Tasks = new List<Task>();
}
```

The hierarchy itself is the domain model: Application -> List<Task>.

10. Layer 4 - ServerFacade

The facade exposes flat functions: GetApplication, CreateTask, UpdateTask, DeleteTask. CreateTask generates OPUSGuid.NewRandom128(), creates a Task, adds it to Application.Instance.Tasks, saves the task and Application through ObjectNetworkEngine, and returns wire data.

11. Layer 5 - ObjectNetworkEngine

The first engine operates on domain objects and converts them to serialized object representation before passing them to DistributedMiddleTier.

12. How Application is persisted

Application does not embed complete serialized Task objects. It persists Task GUID references. Example:

Application GUID AAAA

Task IDs 1111, 2222, 3333

Files: AAAA.bin, 1111.bin, 2222.bin, 3333.bin.

AAAA.bin contains Application and the Task GUID references. Server startup reconstructs real typed Task references.

13. Application serializer

Application serialization writes schema version, Application OPUSGuid, task count, and each Task OPUSGuid. Task serialization

14. Layer 6 - DistributedMiddleTier

IObjectNode exposes:

```
byte[] Read(OPUSGuid id)
```

```
void Write(OPUSGuid id, byte[] data)
```

```
void Delete(OPUSGuid id)
```

DistributedMiddleTier initially forwards to one node. Later it can route by type or GUID hash.

15. Why ObjectNetworkEngine appears twice

The upper ObjectNetworkEngine is server/domain-facing. The lower ObjectNetworkEngine is storage-node-facing. Suggested concrete names are DomainObjectNetworkEngine and StorageObjectNetworkEngine.

16. Layer 7 - storage-side ObjectNetworkEngine

StorageObjectNetworkEngine implements IObjectNode and delegates Read/Write/Delete to IObjectStore.

17. Layer 8 - PhysicalStorage

IObjectStore:

```
bool Exists(OPUSGuid id)
```

```
byte[] Read(OPUSGuid id)
```

```
void Write(OPUSGuid id, byte[] data)
```

```
void Delete(OPUSGuid id)
```

Physical storage semantics are exactly GUID -> byte[]. It knows nothing about Task, Application, Customer, Invoice, ENTER, ...

18. FileSystemObjectStore

FileSystemObjectStore maps OPUSGuid to <guid>.bin and uses temporary files for writes before moving into the final path.

19. Known Application root GUID

The server has a known singleton/root Application GUID so it knows where the object graph begins.

20. Server startup

Physical storage -> load Application -> read task GUID collection -> load each Task -> reconstruct List<Task>.

21. Server composition root

PhysicalStorage -> StorageObjectNetworkEngine -> DistributedMiddleTier -> ObjectSerializer -> DomainObjectNetworkEngine -> ...

22. TCP listener

TcpServer accepts TcpClient instances and queues ServerBinaryProtocol.Process on ThreadPool workers.

23. Client startup

ClientBinaryProtocol is created with server address/port, ClientDomainRuntime is created from it, and runtime.Load() populated.

24. WPF ViewModel

The ViewModel exposes Application.Instance.Tasks and delegates AddTask, SaveTask, DeleteTask and CompleteTask to ClientDomainRuntime.

25. Complete CREATE flow

UI -> ClientDomainRuntime.CreateTask -> ClientBinaryProtocol.CreateTask -> TCP/IP -> ServerBinaryProtocol -> write lock -> ServerFacade.CreateTask -> OPUSGuid.NewRandom128 -> Application.Instance.Tasks.Add -> DomainObjectNetworkEngine.SaveTask -> DistributedMiddleTier.Write -> StorageObjectNetworkEngine.Write -> FileSystemObjectStore.Write -> TaskGUID.bin. Application itself is also saved because the relationship collection changed.

26. UPDATE flow

Client Task -> TaskData -> ClientBinaryProtocol -> TCP -> ServerBinaryProtocol -> ServerFacade.UpdateTask -> find Task in Application

27. DELETE flow

DeleteTask(Guid) -> ServerFacade -> Application.Tasks.Remove(task), delete TaskGUID.bin, update ApplicationGUID.bin.

28. READ/query flow

Because the full domain model is in RAM, queries should hit the in-memory Application graph rather than persistence.

29. Optional typed index

Dictionary<OPUSGuid, Task> can be added as an index for scale, but it is not the domain model.

30. Resulting object architecture

Client memory: Application.Instance -> Tasks.

Protocol: DTO/binary.

Server memory: Application.Instance -> Tasks.

Persistence: ApplicationGUID.bin containing Task GUID references plus one file per Task.

31. Eight layers final responsibilities

- 1 ClientDomainRuntime - client Application singleton and Task graph
- 2 ClientBinaryProtocol - client operations to binary TCP requests
- 3 ServerBinaryProtocol - decoding, locking, dispatch
- 4 ServerFacade - flat application functions/business operations
- 5 ObjectNetworkEngine - server domain objects to serialized objects
- 6 DistributedMiddleTier - routing
- 7 ObjectNetworkEngine - storage-node object service
- 8 PhysicalStorage - OPUSGuid -> byte[] persistence

32. Key OPUS.NET separation

Application-specific ServerFacade sits above reusable OPUS.NET infrastructure. ENTER, MEL and GameX can have their own facade.

33. Recommended interfaces

IObjectStore, IObjectNode and IObjectSerializer<T> separate serialization, distribution and persistence.

34. Final architecture

WPF UI -> ClientDomainRuntime -> ClientBinaryProtocol -> TCP/IP -> ServerBinaryProtocol -> ServerFacade -> Domain ObjectNetworkEngine

The TODO application exercises client domain model, server-private domain model, singleton/root Application, typed hierarchical collections, GUID relationships, flat facade functions, custom binary protocol, reader/writer locking, in-memory server model, distributed object routing, serialization/storage separation, and GUID-to-BLOB persistence.

3. USER

based on the above give me the ClientDomainRuntime classes as Application and Task using the newest variety of OPUSGuid in c#

4. ASSISTANT

Using the latest OPUSGuid conclusion from our earlier debugging - where OPUSGuid.NewRandom128() produces a valid non-zero value and the property itself should simply store that value - the client-side domain model can stay very small.

Application:

```
using System.Collections.ObjectModel;
```

```
namespace OPUS.TODO.ClientDomain
{
```

```
public sealed class Application
```

```
{
```

```
private static readonly Application _instance =
```

```
    new Application();
```

```
public static Application Instance
```

```
{
    get { return _instance; }
}
```

```
public OPUSGuid Id { get; internal set; }
```

```
public ObservableCollection<Task> Tasks { get; private set; }
```

```
private Application()
```

```

        {
            Id = OPUSGuid.Empty;
            Tasks = new ObservableCollection<Task>();
        }
    }
}

Task:

namespace OPUS.Todo.ClientDomain
{
    public sealed class Task
    {
        public OPUSGuid Id { get; internal set; }
        public string Title { get; set; }
        public string Description { get; set; }
        public bool IsCompleted { get; set; }

        internal Task(OPUSGuid id)
        {
            Id = id;
            Title = string.Empty;
            Description = string.Empty;
            IsCompleted = false;
        }

        internal Task(
            OPUSGuid id,
            string title,
            string description,
            bool isCompleted)
        {
            Id = id;
            Title = title;
            Description = description;
            IsCompleted = isCompleted;
        }
    }
}

```

For client-side objects arriving from the server, use the GUID supplied by the server and add it to Application.Instance.Tasks. The client-side Task should normally not generate its own ID. The authoritative ID should be generated on the server with OPUSGuid.NewRandom128() and returned to the client.

Prefer public OPUSGuid Id { get; internal set; } so ClientDomainRuntime can reconstruct objects received from the server wh

5. USER

now give me an implementation of client-side Task object down through the ClientBinaryProtocol into the ServerBinaryProtocol using a non-scheme direct binary serializer on the client-side and a deserializer on the server-side

6. ASSISTANT

For OPUS.NET I would make this a direct binary object transfer: the client-side Task serializes its fields directly into a byte stream, ClientBinaryProtocol wraps that byte array in an operation frame, and ServerBinaryProtocol reconstructs a server-side Task by reading the fields in exactly the same order.

There is no XML, JSON, reflection serializer, schema registry, DTO, or BinaryFormatter involved.

Wire path:

Client Task -> Direct binary serialization -> ClientBinaryProtocol -> TCP/IP -> ServerBinaryProtocol -> Direct binary deser

Client-side Task:

```

using System;
using System.IO;
using System.Text;

namespace OPUS.Todo.ClientDomain
{
    public sealed class Task

```



```

    {
        public OPUSGuid Id { get; internal set; }
        public string Title { get; set; }
        public string Description { get; set; }
        public bool IsCompleted { get; set; }
    }

internal Task(OPUSGuid id)
{
    Id = id;
    Title = string.Empty;
    Description = string.Empty;
    IsCompleted = false;
}

internal Task(OPUSGuid id, string title, string description, bool isCompleted)
{
    Id = id;
    Title = title;
    Description = description;
    IsCompleted = isCompleted;
}

public byte[] Serialize()
{
    using (MemoryStream stream = new MemoryStream())
    using (BinaryWriter writer = new BinaryWriter(stream, Encoding.UTF8))
    {
        writer.Write(Id.ToByteArray());
        WriteString(writer, Title);
        WriteString(writer, Description);
        writer.Write(IsCompleted);
        writer.Flush();
        return stream.ToArray();
    }
}

private static void WriteString(BinaryWriter writer, string value)
{
    if (value == null)
    {
        {
            writer.Write(-1);
            return;
        }

        byte[] bytes = Encoding.UTF8.GetBytes(value);
        writer.Write(bytes.Length);
        writer.Write(bytes);
    }
}

```

Task binary layout:

16 bytes Id
 4 bytes Title byte length
 N bytes Title UTF-8
 4 bytes Description byte length
 N bytes Description UTF-8
 1 byte IsCompleted

Operation codes:

```

public enum OperationCode : byte
{
    CreateTask = 1,
    UpdateTask = 2,
    DeleteTask = 3,
    GetApplication = 4
}

```

OPUS.NET request frame:

BYTE LockMode
 BYTE OperationCode
 INT32 PayloadLength
 BYTE[] Payload

LockMode 0=read, 1=write.

ClientBinaryProtocol uses TcpClient, writes writeOperation, operation, payload length and payload, then reads success, response

Server-side Task:

```
namespace OPUS.TODO.ServerDomain
{
```

public sealed class Task

```
{
    public OPUSGuid Id { get; private set; }
    public string Title { get; set; }
    public string Description { get; set; }
    public bool IsCompleted { get; set; }
```

internal Task(OPUSGuid id, string title, string description, bool isCompleted)

```
{
    Id = id;
    Title = title;
    Description = description;
    IsCompleted = isCompleted;
}
}
```

ServerTaskDeserializer reads exactly 16 GUID bytes, reconstructs OPUSGuid.FromByteArray, reads Title, Description, IsCompleted

ServerBinaryProtocol reads the OPUS.NET request header, validates payload size, acquires ReaderWriterLockSlim based on the v

Important server distinction: the deserialized object is incoming state. ServerFacade should find the authoritative ServerTask in Application.Instance.Tasks by OPUSGuid and copy permitted fields into it, then persist the authoritative task. It should not replace the authoritative object instance.

For CREATE, the client should send only creation data such as Title and Description. The server should generate OPUSGuid.NewRandom128(), create the authoritative Task, persist it, serialize the resulting Task and return it to the client.

The direct serializer contract is:

```
[16-byte OPUSGuid]
[Int32 + UTF8 Title]
[Int32 + UTF8 Description]
[Boolean IsCompleted]
```

Client and server agree on this exact field order.

7. USER

give me the source code in C# on the method OPUSGuid.FromByteArray

8. ASSISTANT

Assuming the newest OPUSGuid internally wraps a standard System.Guid, FromByteArray should be:

```
public static OPUSGuid FromByteArray(byte[] bytes)
{
    if (bytes == null)
        throw new ArgumentNullException(nameof(bytes));

    if (bytes.Length != 16)
        throw new ArgumentException(
            "An OPUSGuid must contain exactly 16 bytes.",
            nameof(bytes));

    return new OPUSGuid(new Guid(bytes));
}
```

This assumes:

```
public OPUSGuid(Guid value)
{
    _value = value;
}
```

and:

```
public byte[] ToByteArray()
{
    return _value.ToByteArray();
}
```

Round trip:

```
OPUSGuid original = OPUSGuid.NewRandom128();
byte[] bytes = original.ToByteArray();
OPUSGuid restored = OPUSGuid.FromByteArray(bytes);
Console.WriteLine(original == restored);
```

The result should be True. FromByteArray must reconstruct the exact 128-bit value and must not generate a new GUID.

9. USER

ObjectNetworkEngine now has an instance of ServerApplication with one ServerTask in the collection. What is the facade/front

10. ASSISTANT

The front/facade of DistributedMiddleTier should not expose ServerApplication or ServerTask directly. Its boundary should be generic and object-network-oriented.

At that point, ObjectNetworkEngine has already translated the server domain model into persistent object units. The next lower layer should receive:

WriteObject(OPUSGuid id, byte[] data)

ReadObject(OPUSGuid id)

DeleteObject(OPUSGuid id)

Recommended interface:

```
public interface IDistributedMiddleTier
{
    byte[] ReadObject(OPUSGuid id);
```

void WriteObject(

OPUSGuid id,

byte[] serializedObject);

void DeleteObject(OPUSGuid id);

bool ObjectExists(OPUSGuid id);

}

Concrete DistributedMiddleTier delegates to an IObjectNetworkNode. The architectural boundary is:

ServerFacade -> ObjectNetworkEngine (understands ServerApplication/ServerTask and converts objects to OPUSGuid + byte[]) ->

For a ServerApplication AAAAAAAA containing ServerTaskBBBBBBBB, the upper ObjectNetworkEngine serializes them as two distinct objects. The Application bytes contain its ID, Task count and Task referenceBBBBBBBB. The Task bytes contain its ID, Title, Description and IsCompleted.

It calls:

```
_distributedMiddleTier.WriteObject(application.Id, serializedApplication);
_distributedMiddleTier.WriteObject(task.Id, serializedTask);
```

A richer ObjectWriteRequest can later contain ObjectId, Data and ObjectTypeId for routing.

Responsibility split:

Upper ObjectNetworkEngine understands objects.

DistributedMiddleTier understands placement/routing.

Lower ObjectNetworkEngine understands object storage operations.

PhysicalStorage understands GUID -> bytes.

In the single-machine version DistributedMiddleTier can simply delegate to StorageObjectNetworkEngine. Later the delegation can become router.ResolveNode(id).

The facade/front should therefore be a generic object-addressed API operating on OPUSGuid + byte[], not on ServerApplication or ServerTask.

11. USER

now i need the source code in C# for the serializer of ServerTask at the ObjectNetworkEngine

12. ASSISTANT

At the ObjectNetworkEngine boundary, serialize ServerTask into a compact deterministic byte array containing only its persisted fields.

Assuming ServerTask:

```
public sealed class ServerTask
{
    public OPUSGuid Id { get; private set; }
    public string Title { get; set; }
    public string Description { get; set; }
    public bool IsCompleted { get; set; }
```

public ServerTask(OPUSGuid id, string title, string description, bool isCompleted)

```
    {
        Id = id;
        Title = title;
        Description = description;
        IsCompleted = isCompleted;
    }
}
```

Serializer:

```
using System;
using System.IO;
using System.Text;

namespace OPUS.ObjectNetworkEngine
{
```

public static class ServerTaskSerializer

```
{
```

public static byte[] Serialize(ServerTask task)

```
{
```

if (task == null)

```
    throw new ArgumentNullException(nameof(task));
```

```
    using (MemoryStream stream = new MemoryStream())
```

```
    using (BinaryWriter writer = new BinaryWriter(stream, Encoding.UTF8))
```

```
    {
```

```
        writer.Write(1);
```

```
        byte[] idBytes = task.Id.ToByteArray();
```

if (idBytes.Length != 16)

throw new InvalidOperationException(

```
    "OPUSGuid must serialize to exactly 16 bytes.");
```

```
    writer.Write(idBytes);
```

```
    WriteString(writer, task.Title);
```

```
    WriteString(writer, task.Description);
```

```
    writer.Write(task.IsCompleted);
```

```
    writer.Flush();
```

```
    return stream.ToArray();
```

```
    }
```

```
}
```

private static void WriteString(BinaryWriter writer, string value)

```
{
```

if (value == null)

```
{
```

```
    writer.Write(-1);
```

```
    return;
```

```

    }

    byte[] bytes = Encoding.UTF8.GetBytes(value);
    writer.Write(bytes.Length);
    writer.Write(bytes);
}
}
}

```

Binary layout:

INT32 Object format version
 BYTE[16] ServerTask.Id
 INT32 Title byte length
 BYTE[N] Title UTF-8
 INT32 Description byte length
 BYTE[N] Description UTF-8
 BYTE IsCompleted

Inside ObjectNetworkEngine:

```

public void SaveTask(ServerTask task)
{
    if (task == null)
        throw new ArgumentNullException(nameof(task));

```

byte[] serializedTask =

```

    ServerTaskSerializer.Serialize(task);
    _distributedMiddleTier.WriteObject(
task.Id,
        serializedTask);
}

```

Keep this persistence serializer separate from the client/server network serializer. ClientBinaryProtocol format is a common

13. USER

i now have WriteObject on the facade DistributedMiddleTierCRUD. The next lower layer is called ObjectNetworkEngine with the facade ObjectNetworkEngineASIC. Give me the contract to translate from DistributedMiddleTier into this new facade.

14. ASSISTANT

Given your current naming, make ObjectNetworkEngineASIC the generic lower-layer contract that DistributedMiddleTierCRUD.WriteObject(...) calls after it has decided where the object belongs.

At this boundary, the distributed layer should already have reduced the operation to OPUSGuid + byte[].

Contract:

```

public interface ObjectNetworkEngineASIC
{
    void WriteObject(
        OPUSGuid objectId,
        byte[] serializedObject);

    byte[] ReadObject(
        OPUSGuid objectId);

    void DeleteObject(
        OPUSGuid objectId);

    bool ObjectExists(
        OPUSGuid objectId);
}

DistributedMiddleTierCRUD delegates to this interface:

```

```

public sealed class DistributedMiddleTierCRUD
{
    private readonly ObjectNetworkEngineASIC _objectNetworkEngine;

public DistributedMiddleTierCRUD(
ObjectNetworkEngineASIC objectNetworkEngine)
    {
        _objectNetworkEngine = objectNetworkEngine;
    }

public void WriteObject(OPUSGuid objectId, byte[] serializedObject)
    {
        _objectNetworkEngine.WriteObject(objectId, serializedObject);
    }

public byte[] ReadObject(OPUSGuid objectId)
    {
        return _objectNetworkEngine.ReadObject(objectId);
    }

public void DeleteObject(OPUSGuid objectId)
    {
        _objectNetworkEngine.DeleteObject(objectId);
    }

public bool ObjectExists(OPUSGuid objectId)
    {
        return _objectNetworkEngine.ObjectExists(objectId);
    }
}

```

The lower ObjectNetworkEngine implements ObjectNetworkEngineASIC and delegates to PhysicalStorageCRUD.

Architecture:

Upper ObjectNetworkEngine
understands ServerTask / ServerApplication
serializes them

|
v

DistributedMiddleTierCRUD

WriteObject(OPUSGuid, byte[])
decides distribution / destination

|
v

ObjectNetworkEngineASIC

WriteObject(OPUSGuid, byte[])
ReadObject(OPUSGuid)
DeleteObject(OPUSGuid)
ObjectExists(OPUSGuid)

|
v

lower ObjectNetworkEngine

|
v

PhysicalStorage

Do not pass ServerTask through ObjectNetworkEngineASIC. By this point type-specific work is finished. The boundary is OPUSG

15. USER

give me a full chronological without omissions pdf document for download from the entire above conversation